

[ACJC/2022/Q2]

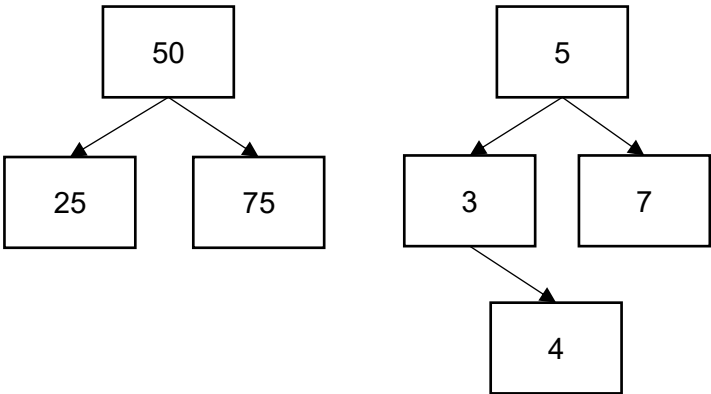
- 2
- Binary search trees can be stored in a two-dimensional array, where each row of the array represents a node and contains the following information:
- the data (an integer) stored within that node;
 - the left pointer, an integer which is the index of the left child of that node;
 - the right pointer, an integer which is the index of the right child of that node.

Unused rows within the array are stored as a free list, a linked list where each node's left child is the next item in the list.

For instance, the diagram below shows how two trees are stored within an array.

Index	Data	Left	Right
0	50	2	6
1	5	8	3
2	25	None	None
3	7	None	None
4	4	None	None
5	None	9	None
6	75	None	None
7	None	5	None
8	3	None	4
9	None	None	None

Free list pointer: 7
Tree 1 pointer: 0
Tree 2 pointer: 1



Task 2.1

The array is initialised with `n` rows, where `n` is determined by the user when the array is created.

Upon initialisation, the data in each row is `None`, and the left pointer in each row points to the row immediately below it. The left pointer of the final row is `None`. The free list pointer points to the first row.

Write code to create an `Array` class as described above. There should be accessor and mutator methods for the free list pointer, as well as the data, left and right pointers for each row. [9]

[ACJC/2022/Q2 Cont']

Task 2.2

A BST (Binary Search Tree) is initialised using an existing `Array` object as an argument. It is initialised as an empty tree, with a pointer value of `None`.

Create a `BST` class.

[2]

Write code for the following methods:

- `add_node(data)` takes the first available node from the free list, inserts `data` as the node's data, and adds the node to the appropriate location in the BST. If the free list is empty, an appropriate message should be output. It may be assumed that the new node does not have the same data as any of the existing nodes in the BST. [10]
- `search_tree(data)` searches the BST for a node with `data`, and returns the index of that node within the `Array`. If the data does not exist within the BST, an appropriate message should be output. [6]
- `in_order_traversal()` returns the data in the nodes of the BST, following an in-order traversal. [5]

Your methods should demonstrate encapsulation of the `Array` object.

Download your program code and output for Task 2 as

`TASK2_<your name>_<centre number>_<index number>.ipynb`